

Aurum Rails — Submission

The Stablecoins Commerce Stack Challenge · Ignite × Circle × Arc

1. Project Title

Aurum Rails — Transparent UAE → Global Cross-Border Payments on Arc

2. Track

Track 1 — Best Cross-Border Payments & Remittances Experience (UAE → Global)

3. Circle Account Email

my5757980@gmail.com

4. Links

Demo Application URL	https://aurum-rails-uae-remittance.vercel.app
GitHub Repository	https://github.com/my5757980/aurum-rails-uae-remittance
Demo Video	https://streamable.com/wutq5h
Live proof (on-chain)	https://testnet.arcscan.app/tx/0x65e999db15e4c6c718312a7c8b29102c62b55fe310d114176e74ede171622acb

To try it: open the app and choose **"Try the demo — no sign-up"**. That skips the email step, which matters because the free Supabase tier rate-limits signup email. Everything after that is live: the quote, the transfer, and the explorer link.

The transaction hash above was produced **by the deployed application**, not locally.

5. Description

Aurum Rails is a remittance product for the UAE → Global corridor, built on Circle's stablecoin stack and Arc. A sender in Dubai chooses a recipient, sees **every cost before committing**, confirms once, and watches a real USDC transfer settle on Arc Testnet in seconds — with a public transaction anyone can verify.

The insight the product is built on

Most remittance pitches open with "fees are too high." **For the UAE that is false**, and we refuse to claim it.

The World Bank's *Remittance Prices Worldwide* puts the average cost of sending USD 200 from the UAE at **under 3.5%** — well below the 6.62% global average. UAE exchange houses advertise **AED 15–26 flat**. Building on a "we're cheaper" story would be both dishonest and trivially falsifiable by anyone who knows the corridor.

The real pain is **where the cost hides and what the sender cannot see**:

Pain point	What the sender actually experiences
Hidden FX spread	The advertised fee is AED 15. The real cost is the margin baked into the AED→INR rate, never shown beside a mid-market reference.
No live status	A reference number and "2–3 business days". The money is a black box between correspondent banks.
Banking hours	Send on a Friday evening in Dubai; nothing moves until the following week.
Unpredictable landing	The recipient often does not know the exact amount until it arrives.

So we compete on **transparency and time-to-settlement**, and we prove both on-chain:

- The **FX spread is a named line item, displayed even when it is 0.00%** — showing the zero is what proves the line is real.
- Every exchange rate carries its **source and timestamp**.
- Every settled payment carries a **public explorer link**.
- Everything simulated is **labelled as simulated**, in the product and in this document.

6. Working MVP

Live at <https://aurum-rails-uae-remittance.vercel.app> — Next.js on Vercel, with the API routes running as serverless functions and PostgreSQL (Supabase) behind them. Frontend and backend are one deployment; there is no separate backend service.

All of the following was executed against the deployed application and verified, not described from a plan.

What deploying taught us — three bugs local testing could not find

Worth stating because it shaped the architecture. Deploying early and testing against the real host surfaced three defects, all with the same root cause: **code that assumed the process keeps living after it responds**. Serverless freezes the function the instant the response is sent.

1. **Status was driven by a background timer** started after responding. It never ran in production. Status is now advanced on demand — the UI's own polling moves the state machine, which is one code path that is correct on both a long-running server and a serverless one.
2. **The in-memory store is per-instance**, and requests spread across instances, so a quote created on one was absent on another. Reads now fall back to PostgreSQL. The idempotency check got the same treatment,

and that one matters most: an in-memory-only check on a retry landing elsewhere would happily pay someone twice.

3. **Database writes were fire-and-forget**, so the write carrying the Circle transaction id died with the frozen function. Every payment sat at `INITIATED` forever while the UI showed a spinner. **This one was only caught by testing the deployed URL** — the build passed, the local run passed, and the product was still broken.

The third is the reason this section exists. "It builds" and "it works locally" are not evidence that a payments product works.

Consumer remittance (hero flow)

Sign in → choose recipient → enter amount → full cost disclosure → confirm once → live status timeline → receipt with explorer link.

Measured: `INITIATED → SUBMITTED → SETTLING → SETTLED → DELIVERING → DELIVERED` in **3.6–7.4 seconds** end to end. The explorer reports the transaction **confirmed on chain in ≤ 0.51 s** — the remaining wall-clock is our own API polling, not the network.

Business / freelancer batch payouts

Select several contractors, one aggregate quote, one authorisation. Items settle and fail **independently** — a single failure never blocks the rest. Per-payee invoice references carried through to individual receipts.

Verified: a run of 3 contractors executed 3/3, each as its own real Arc transaction.

Cross-chain delivery

A recipient can be paid on another network. The payment settles on Arc, then bridges via **CCTP v2 / App Kit**.

Verified: `Arc_Testnet → Base_Sepolia`, `transferSpeed: FAST`, all four steps — `approve → burn → fetchAttestation → mint` — completed in 35.2 s with a real transaction hash on each side.

Recipient experience

The recipient opens a single link. **No account, no app, no wallet, and not one word of crypto vocabulary.** This is enforced by an automated check in the build: user-facing copy is scanned and a banned term fails the build.

Correctness guarantees, tested

Behaviour	Result
Double-tap Send / repeated Idempotency-Key	Exactly one transfer created
Missing Idempotency-Key	Rejected, 400
Expired quote (60 s) executed	Rejected, 409 — a stale rate is never used
Amount above / below demo limits	Rejected, 422, plain-language message
Insufficient balance	Blocked before submission, never a raw chain error
Auth gate	Protected routes 401; the recipient claim link stays public by design
Responsive	50/50 page-width combinations from 320 px to 1920 px — zero horizontal overflow, zero clipped text
Build	Production build exit 0 · 31/31 unit tests · copy check pass

7. Products Used

Circle products used on Arc — selected from the challenge list:

Product	Used	Notes
USDC	✓	Settlement rail for every payment
Wallets	✓	Developer-Controlled, one per sender and recipient
Gateway	✓	Treasury balance; deposits not made — see §11
CCTP / Bridge Kit	✓	Via App Kit; Arc → Base Sepolia verified
USYC	—	Not applicable to this use case
StableFX	—	Relevant to AED↔USD but access is gated
Nanopayments	—	Not applicable to this use case

Four products doing real work. We did not add products we do not need in order to raise a count — the reasoning is in §11.

Circle product	What it does in this project	Where in the code
USDC on Arc Testnet	Settlement rail for every payment. USDC is also the native gas asset on Arc, so network cost and payment share one unit — which is what makes a single honest fee number possible.	<code>lib/chain.ts</code> , <code>lib/money.ts</code>
Circle Wallets (Developer-Controlled)	A wallet per sender and per recipient, provisioned server-side and invisibly. This is what removes seed phrases and browser extensions — the reason a non-crypto-native persona can use this at all.	<code>lib/wallet-service.ts</code>
Circle Gateway	Treasury balance for the business payout surface, with per-chain composition.	<code>lib/treasury.ts</code> , <code>app/api/treasury/route.ts</code>
CCTP v2 / App Kit	Cross-chain USDC delivery when a recipient is paid on another network. Verified Arc → Base Sepolia.	<code>lib/bridge.ts</code> , <code>scripts/spike-bridge.ts</code>

Stack: Next.js 16 (App Router) · React 19 · TypeScript strict · Tailwind + shadcn/ui · PostgreSQL (hosted on Supabase) · Vitest.

A note on Gateway, stated plainly. Circle Gateway reports funds *deposited into the Gateway Wallet contract*, and this build has not made that deposit. Rather than invent a unified figure, the treasury falls back to **real Arc balances** and the UI labels the source **"ARC ONLY"** with a sentence explaining why. We would rather show a smaller true number than a larger invented one — particularly in a payments product.

8. Architecture Diagram

A rendered Mermaid version is in the repository README. Text equivalent:



Two engineering decisions worth defending

1. Arc's dual-interface decimal model is a compile error, not a hope. Arc exposes one pool of funds through an **18-decimal native view** (gas) and a **6-decimal ERC-20 view** (balances, transfers), where `1e18 native == 1e6 ERC-20`. Mixing them is a **silent 10^{12} error in a payment path** — the worst class of bug a money

product can have, and invisible in testing at small amounts. We made the two types physically unassignable to one another, verified by `@ts-expect-error` assertions in the test suite. Money is `bigint` minor units throughout; `number` never touches a monetary value.

2. Status is an append-only event log, never the chain. A transfer's state is the latest recorded `status_event`, never re-derived by querying Arc at render time. Transitions are recorded, not inferred, and a `FAILED` event **cannot exist without a reason** — enforced by a database constraint, not a convention.

9. What Is Real vs. Simulated

Stating this precisely is part of the submission, not a disclaimer attached to it.

Element	Status
USDC transfer on Arc Testnet	✔ Real — verifiable on <code>testnet.arcsan.app</code>
Wallet creation and custody (Circle DCW)	✔ Real
Network cost	✔ Real — observed
Service fee	✔ Real — charged in the flow
Exchange rate	✔ Real — live, with source and timestamp shown
Settlement timing	✔ Real — measured, shown as a live counter
Cross-chain bridge (Arc → Base Sepolia)	✔ Real — four steps, real hashes
Local-currency payout (INR / PKR / PHP)	⚠ Simulated — labelled everywhere it appears
AED pay-in	⚠ Not built — the sender's AED balance is conceptual
KYC / AML / sanctions screening	✖ Not implemented — a real product could not launch without it

Amounts on screen are 1:1 with what moves on chain. We deliberately rejected a display scale factor: if the interface said AED 100 while a fraction of a dollar moved, the number the user sees would not be the number that settled. Small real amounts, perfectly reconcilable against the explorer.

Testnet only. Educational demo. No real funds.

10. Documentation

Everything below is in the repository.

Document	Contents
README.md	Problem, architecture diagrams, Circle product → file map, real-vs-simulated table, 10-minute setup
docs/PROVENANCE.md	Forked from circlefin/arc-commerce @ 1a3a5e0 (Apache-2.0), with the full delta and findings on arrival
docs/circle-feedback-notes.md	Ten reproducible findings with measurements
docs/demo-script.md	Narrated video script, timed
specs/001-uae-global-remittance/	Constitution, spec, plan, research, data model, OpenAPI contracts, task list
supabase/migrations/	Schema with the constraints that matter enforced <i>as</i> constraints

Built under a spec-driven workflow: a written constitution set non-negotiable principles (Arc Testnet only, real flows over mockups, radical transparency, consumer-grade UX, secrets discipline) **before** any code, and every subsequent decision was checked against it.

11. Circle Product Feedback

Specific and reproducible. Written to be useful rather than flattering.

Why we chose these products for this use case

USDC on Arc — because the corridor problem is timing, and Arc removes it. The pain we set out to fix is that a sender cannot see where their money is for two days, and that a Friday-evening transfer sits until the following week. A chain with deterministic sub-second finality does not merely make that faster — it removes the category of problem. That USDC is *also* the native gas asset matters more than it sounds: fee and payment share one unit, which is the only reason a single honest fee number is expressible at all. On a chain with a separate gas token, our transparency promise would have needed an asterisk.

Circle Wallets (Developer-Controlled) — because the persona decides the architecture. Our user is a site supervisor in Dubai who has no reason to learn what a wallet is. Any design requiring a seed phrase or a browser extension fails before it starts. DCW is what makes a seedless, extension-free, server-orchestrated flow possible, and it is why the recipient can be paid without ever holding an account. We chose Developer-Controlled over User-Controlled specifically so batch payouts could execute without per-item user signing.

Circle Gateway — because a business must promise a landed amount before it pays. The B2B half of this track is a company paying contractors across several countries. The failure mode is discovering mid-run that liquidity sits on the wrong chain. A unified balance is the right primitive for quoting a run before authorising it. (We did not complete a Gateway *deposit*, so our treasury reads Arc-only and says so — see below.)

CCTP / App Kit — because "global" cannot mean "only if they are on our chain." A UAE → Global product that requires every recipient to be on Arc is not global. Canonical burn-and-mint was the only option we would accept: wrapped or bridged representations would undermine the same guarantee — that what settles is what was sent — that the rest of the product is built on.

Products we deliberately did not use. USYC (yield on idle treasury float) and Nanopayments solve problems this product does not have; adding them to inflate a count would have been dishonest. StableFX is genuinely

relevant to AED↔USD settlement and we would reach for it next, but access is gated and we would rather ship four products doing real work than five where one is decoration.

What worked well

- **ARC-TESTNET in Developer-Controlled Wallets is genuinely turnkey.** Wallet set → wallets → first real transfer took under an hour from a cold start.
- **Sub-second finality is real, and under-advertised.** The explorer reported "**Confirmed within ≤ 0.51 secs.**" For a remittance product, "arrives while you are still looking at the screen" is the single most compelling property of the chain.
- **USDC as the native gas asset is a real UX unlock.** Not having to explain "you also need a second token for gas" is the difference between a product a construction supervisor in Dubai can use and one they cannot.
- **The `circlefin/skills` repository is the best documentation Circle ships.** `use-arc`, `use-gateway` and `bridge-stablecoin` were denser and more accurate than the docs site.
- **Mandatory UUID v4 `idempotencyKey` on every mutation is the right default.** It let our exactly-once requirement and Circle's become the *same* key rather than two that can disagree.

Friction, in the order we hit it

1. **⚠️ Arc's dual-interface decimal model needs a far louder warning.** One pool of funds, 18-decimal native view for gas, 6-decimal ERC-20 view for everything else. Documented in roughly one sentence. Mixing them is a silent 10^{12} error in a payment path. This deserves a prominent callout in the Arc quickstart, and ideally SDK-level types that make the two views non-interchangeable. Every team will otherwise reinvent this defence.
2. `createTransaction` **takes `amount` (singular) as a string array** — `amount: ["0.01"]`. We wrote `amounts` first. TypeScript caught it before a credentialed call, which was a genuine save, but the singular name for an array field is a trap.
3. `registerEntitySecretCiphertext({ recoveryFileDownloadPath })` **wants a directory**, not a file path. Passing a filename fails with `Invalid Directory`. The parameter name reads like a file path.
4. **"The secret for this entity has already been set" is accurate but not actionable.** The entity secret is **account-scoped**, not project-scoped. Arriving from a second project with a fresh API key, the error does not say the one thing you need to hear: *reuse your existing entity secret — it works with any API key on this account*. This cost us more time than any other single issue, and it is a one-line documentation fix.
5. `listWallets` **does not guarantee ordering.** We took `wallets[0]` as the sender; the order changed between runs and the address we had funded silently became the *recipient*. An explicit "ordering is not stable" note, or a sort parameter, would prevent this.
6. **Gateway's empty-balance response is indistinguishable from "unsupported chain."** Querying balances for a wallet holding USDC on Arc returned nothing usable, because the funds were not *deposited into the Gateway Wallet contract*. That is correct behaviour — but the response gives no way to tell "you have no deposits" from "Gateway is not available here," and the latter is exactly the wrong conclusion to draw. We nearly drew it.
7. **CCTP FAST mode took 35.2 s against a documented 8–20 s.** Not a failure, but products quote arrival estimates against that range. A stated p50/p95, or a note that Arc may sit outside it, would help.

8. **The faucet limit is widely misreported.** Third-party sources claim ~1 USDC/day; the faucet itself gives **20 USDC per address every 2 hours**. We designed an entire cent-denominated demo strategy around the wrong figure before checking the source.

On the `arc-commerce` sample

9. **It does not typecheck on a clean clone.** Three committed `components/ui/*` files import `@radix-ui/react-checkbox`, `cmdk` and `react-hook-form`, none of which are in `package.json`. A CI typecheck on the sample would catch this.
10. **The sample uses floating-point money.** `convertToSmallestUnit` does `parseFloat(amount) * 1e6`, and `amount_usdc` moves through the API as a JS `number`. For a *payments* sample this is a pattern developers will copy. `bigint` minor units would set a far better example.
11. **The sample's payment model inverts the value proposition of Developer-Controlled Wallets.** `arc-commerce` initiates payment from a **browser wallet** (wagmi/WalletConnect) and uses the Circle wallet only to receive. But the entire pitch of DCW — no seed phrase, no extension, no key management — argues for *server-initiated* transfers. We replaced the whole payment-initiation path to build for a non-crypto-native persona. **A sample showing DCW → DCW would be far more representative of what DCW is actually for**, and is the single change that would most help the next team building on this track.
-

12. Summary

A working, verifiable cross-border payment product for the UAE corridor, built on four Circle products doing real work on Arc Testnet.

We did not claim to be cheaper than the incumbents, because the data does not support it. We did not show a Gateway balance we had not earned. We labelled every simulated element, in the product and here.

Not cheaper than the exchange house — but you can see exactly what happens to your money, and it arrives while you are still looking at the screen.

Aurum Rails · Ignite × Circle × Arc · Track 1 · Arc Testnet only, educational demo.